Домашнее задание: транспортные протоколы на примере Discord

1. Какие протоколы использует Discord

Discord — мессенджер с текстовыми чатами, голосовыми каналами и демонстрацией экрана. Клиент держит сразу несколько каналов связи, и для каждого выбран свой протокол.

Канал	**Прикладной протокол**	**Транспорт**	**Для чего**
REST API	HTTPS	TCP	Отправка сообщений, загрузка истории, настройки сервера и профиля
Gateway	WebSocket (wss://gateway.discord.gg), данные в JSON или бинарном ETF, сжатие zlib/zstd	TCP	События в реальном времени: новые сообщения, «печатает…», онлайн-статусы
Voice Gateway	WebSocket	TCP	Сигнализация звонка: выбор сервера, ключи шифрования, кто говорит
Голос и видео	RTP + кодек Opus (48 кГц, стерео), шифрование AES-256-GCM / XChaCha20; в браузере — WebRTC	**UDP**	Сам звук, камера, демонстрация экрана
CDN	HTTPS	TCP	Картинки, вложения, аватарки, эмодзи

Как происходит подключение к голосовому каналу:

1. Клиент через основной Gateway сообщает, что заходит в голосовой канал, и получает адрес голосового сервера.
2. Открывает с ним WebSocket (Voice Gateway) и получает UDP-адрес, порт и режимы шифрования.
3. Через UDP узнаёт свой внешний IP и порт (IP Discovery — нужно из-за NAT).
4. Дальше голос идёт по UDP на сервер Discord (SFU), а тот рассылает его остальным участникам. С 2026 г. звонки дополнительно защищены

сквозным шифрованием (протокол DAVE).

Итог: текст и события идут по **TCP** (HTTPS + WebSocket), голос и видео — по **UDP** (RTP).

2. Почему выбран именно такой стек

У текста и голоса противоположные требования, поэтому одного протокола на всё не хватает.

Текст — HTTPS (REST) поверх TCP. Сообщение нельзя потерять или перепутать по порядку, а задержка в сотню миллисекунд незаметна. TCP даёт надёжность, TLS — шифрование. REST работает в любом браузере и удобен для сторонних ботов.

События — WebSocket. Клиенту нужно узнавать о новом сообщении сразу, а не спрашивать сервер каждую секунду. WebSocket держит одно постоянное соединение, по которому сервер сам присылает события, и при этом работает в браузере.

Голос — RTP поверх UDP. Для разговора важна минимальная задержка. Потерянный пакет — это 20 мс звука, кодек Opus его сгладит, и человек не заметит. А пересылать его бессмысленно: пока он дойдёт, момент уже прошёл.

Почему не подходят альтернативы:

Альтернатива	**Почему не подходит**
Голос по TCP	Head-of-Line Blocking: один потерянный пакет задерживает все следующие → звук заикается и отстаёт
Текст по UDP	Сообщения могут потеряться или прийти не по порядку — надёжность пришлось бы писать самим
HTTP-опрос (polling) вместо WebSocket	Лишние запросы и задержка; при миллионах онлайн-пользователей серверы не выдержат
gRPC	В браузере напрямую не работает (нужен gRPC-Web), а для публичного API ботов REST проще
MQTT	Нужен брокер, сообщения не хранятся; протокол заточен под IoT, а не под чат

Альтернатива	Почему не подходит
P2P-звонки без сервера	Нормально для 2 человек, но в канале на 10+ участников каждому пришлось бы слать свой голос всем сразу — не хватит канала. Поэтому Discord гонит голос через свои серверы (SFU)

3. Переход на QUIC / HTTP/3

Сейчас API и события Discord работают поверх TCP. Пользователи предлагали перейти на QUIC ещё в 2018 году, но официального перехода Discord не объявлял.

Что дал бы переход:

- **Мобильность соединения — главный плюс.** Вышел из дома, телефон переключился с Wi-Fi на 4G — сейчас WebSocket рвётся, и звонок переподключается. В QUIC соединение определяется не IP-адресом, а идентификатором, и переживает смену сети.
- **Быстрее запуск и переподключение.** TCP + TLS тратят на рукопожатия 2–3 RTT, а QUIC — 1 RTT, при повторном входе — 0 RTT. Приложение быстрее открывается и после обрыва быстрее возвращается.
- **Нет Head-of-Line Blocking.** Картинки, сообщения и события можно гнать независимыми потоками: потеря пакета в одном не тормозит остальные. Особенно заметно на плохом мобильном интернете.
- **Один протокол вместо двух.** В будущем и события, и голос можно вести в одном QUIC-соединении: надёжные потоки — для событий, ненадёжные датаграммы QUIC — для звука.
- **Шифрование встроено** (TLS 1.3), служебные заголовки тоже скрыты.

Ограничения:

- **UDP блокируют.** Корпоративные сети и часть провайдеров режут или замедляют UDP, так что запасной вариант по TCP всё равно придётся оставить — получатся две версии стека.
- **Больше нагрузка на серверы.** QUIC работает в пользовательском пространстве и шифрует каждый пакет, поэтому тратит больше CPU,

чем TCP из ядра ОС. При миллионах постоянных подключений это заметные деньги.

- **WebSocket придётся менять.** Классический WebSocket живёт на TCP; его аналоги поверх HTTP/3 (например, WebTransport) поддерживаются не всеми браузерами и библиотеками ботов.
- **Для голоса выигрыш маленький.** Голос уже идёт по UDP со своим шифрованием, и переделка рабочей системы на 2,5+ млн одновременных голосовых пользователей — риск ради небольшого улучшения.
- **Сложнее инфраструктура.** Балансировщики, защиту от DDoS и мониторинг надо дорабатывать: зашифрованный QUIC сложнее анализировать и маршрутизировать.

Вывод: больше всего QUIC дал бы мобильному приложению — события и API переживали бы смену сети. Разумно переводить на QUIC сначала REST и CDN с сохранением TCP как запасного варианта, а голос оставить на RTP/UDP.

4. Где UDP лучше TCP: онлайн-шутер Counter-Strike 2

В CS2 сервер постоянно рассылает игрокам состояние матча: позиции, выстрелы, здоровье. Делает он это десятки раз в секунду, по UDP.

Почему UDP:

- **Каждое обновление заменяет предыдущее.** Если пакет с позицией противника потерялся, через несколько миллисекунд придёт новый со свежими данными. Пересылать старую позицию бессмысленно.
- **В TCP потеря одного пакета останавливает все следующие** (Head-of-Line Blocking), пока он не будет дослан. Игрок увидел бы «фриз», а потом рывок противников вперёд. В шутере это проигранная перестрелка.
- **Минимальная задержка.** У UDP нет рукопожатия и ожидания подтверждений. Каждые 10–20 мс пинга видны в игре.
- **Надёжность только там, где нужна.** Важные события (чат, покупка оружия, конец раунда) игра подтверждает и пересылает сама, поверх

UDP. Остальное идёт без гарантий. TCP так разделить трафик не даёт: у него гарантировано всё.

Вывод: в играх реального времени свежие данные важнее полных. Поэтому UDP с выборочной надёжностью на уровне приложения подходит лучше TCP. По той же причине Discord передаёт по UDP голос.